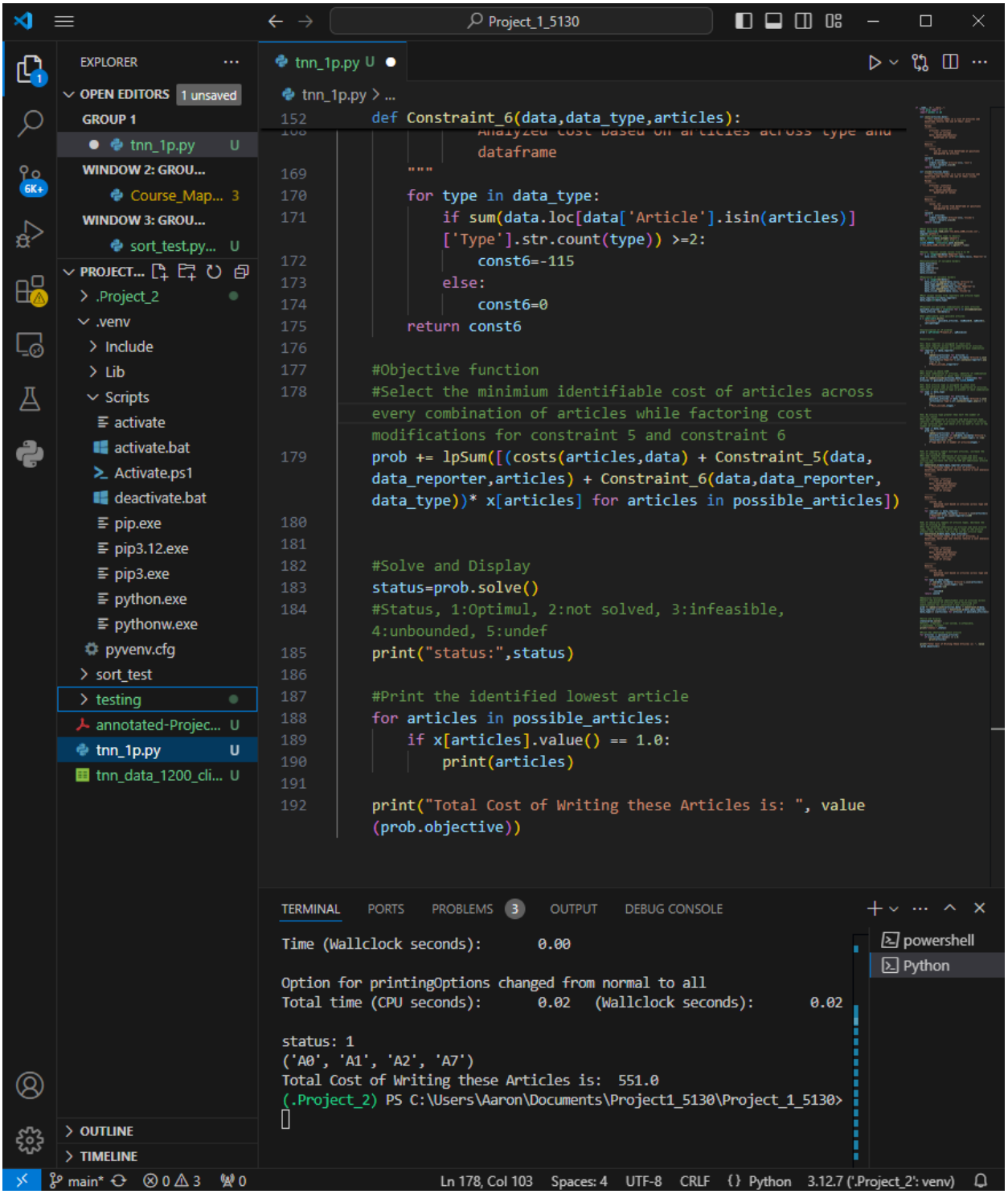
Aaron Heumphreus

Project 2

10/21/2024

CS 5130



```
if __name__ == "__main__":
    from pulp import *
    import pandas as pd

    def costs(articles,data):
        """This function takes in a list of articles and dataframe returns the
sum of their costs
        ----------
        Params
        ----------
            articles: list[str]
                list of strings
            data: dataframe[objects]
                dataframe of values

        ----------
        Returns
        ----------
            value: int
                sum of costs from dataframe at positions designated by articles
        """
        value=0
        for i in articles:
            x=data.loc[data['Article']==i,'Cost']
            value = value+x.iloc[0]
        return (value)

    def clicks(articles,data):
        """This function takes in a list of articles and dataframe and returns
the sum of their clicks
        ----------
        Params
        ----------
            articles: list[str]
                list of strings
            data: dataframe[objects]
                dataframe of values

        ----------
        Returns
        ----------
            value: int
                sum of clicks from dataframe at positions designated by articles
        """
        value=0
```

```
        for i in articles:
            x=data.loc[data['Article']==i,'Clicks']
            value = value+x.iloc[0]
        return (value)


    #Read data from selected CSV
    data: object=pd.read_csv('tnn_data_1200_clicks.csv', engine='python')
    #Reformat dataframe type to objects
    data: object=data.astype('object')
    #Parse click number value from csv title
    CLICK_NUMBER:
int=int(os.path.basename('tnn_data_1200_clicks.csv').split("_")[2])


    #Rename reporter column values from # to R#
    for i in range(len(data['Reporter'])):
        data.loc[i,'Reporter']="R"+str(data.loc[i,'Reporter'])

    #Initialization of variable holders
    data_article=[]
    data_type=[]
    data_reporter=[]
    data_cost=[]
    data_clicks=[]

    #Population of variable holders
    for i in range(len(data)):
        data_article.append(data.loc[i,'Article'])
        data_type.append(data.loc[i,'Type'])
        data_reporter.append(data.loc[i,'Reporter'])
        data_cost.append(data.loc[i,'Cost'])
        data_clicks.append(data.loc[i,'Clicks'])

    #Pull unique values from reporters and article types
    data_reporter=set(data_reporter)
    data_type=set(data_type)


    #Populate all possible combinations of data articles
    possible_articles = [tuple(c) for c in allcombinations(data_article,
len(data))]

    #Set LpVariables with possible articles
    x = LpVariable.dicts(
```

```
        "articles", possible_articles, lowBound=0, upBound=1, cat=LpInteger
    )

    #Initalization of LP problem
    prob = LpProblem("Project_2", LpMinimize)


    #Constraints:

    #C1: Each reporter is assigned at least once
    #For each reporter across each combination of articles, identify if each
reporter is present in each combination
    for reporter in data_reporter:
        prob += (
            lpSum([x[articles] for articles in possible_articles if
data.loc[data['Article'].isin(articles)]['Reporter'].str.contains(reporter).any()
]) >= 1,
            f"Must_include_{reporter}"
        )

    #C2: Clicks is above 1200
    #For each combination of articles, identify if combination of clicks is
greater or equal than clicknumber
    prob += lpSum([clicks(articles,data) * x[articles] for articles in
possible_articles]) >= CLICK_NUMBER

    #C3: Each article type is assigned at least once
    #For each article type across each combination of articles, identify if each
article type is present in each combination
    for type in data_type:
        prob += (
            lpSum([x[articles] for articles in possible_articles if
data.loc[data['Article'].isin(articles)]['Type'].str.contains(type).any()] ) >=
1,
            f"Must_include_{type}."
        )


    #C4: No article type greater than half the number of articles chosen
    #For each combination of articles and each article type, identify the number
of articles and the number of articles of the selected type and check if it is
half or less of the number of articles.
    for type in data_type:
        prob +=(
```

```
            lpSum([x[articles] for articles in possible_articles if
sum(data.loc[data['Article'].isin(articles)]['Type'].str.count(type)) <=
(len(articles)/2)]) ==1,
            f"Type must be <= number of articles{type}."
        )


    #C5: If reporters submit multiple articles, increase the cost of writing by
100
    #For the selected combination of articles and each reporter, check if the
reporter has submitted more than 1 article and increase the cost by 100 per
additional article per reporter
    def Constraint_5(data,data_reporter,articles):
        """This function takes in a list of articles, a dataframe, data_type and
returns returns a cost analysis
        ----------
        Params
        ----------
            articles: list[str]
                list of strings
            data: dataframe[objects]
                dataframe of values
            data_type: list[str]
                list of strings

        ----------
        Returns
        ----------
            const6: int
                Analyzed cost based on articles across type and dataframe
        """
        for reporter in data_reporter:
            const5=sum(data.loc[data['Article'].isin(articles)]['Reporter'].str.c
ount(reporter))*100
            return const5

    #C6: If there are repeats of article types, decrease the cost of writing by
115
    #For the selected combination of articles and each article type, check if
there is more than 1 type of the article present then decrease the cost by 115
per article type.
    def Constraint_6(data,data_type,articles):
        """This function takes in a list of articles, a dataframe, data_type and
returns returns a cost analysis
        ----------
```

```
        Params
        ----------
            articles: list[str]
                list of strings
            data: dataframe[objects]
                dataframe of values
            data_type: list[str]
                list of strings

        ----------
        Returns
        ----------
            const6: int
                Analyzed cost based on articles across type and dataframe
        """
        for type in data_type:
            if
sum(data.loc[data['Article'].isin(articles)]['Type'].str.count(type)) >=2:
                const6=-115
            else:
                const6=0
        return const6

    #Objective function
    #Select the minimium identifiable cost of articles across every combination
of articles while factoring cost modifications for constraint 5 and constraint 6
    prob += lpSum([(costs(articles,data) +
Constraint_5(data,data_reporter,articles) +
Constraint_6(data,data_reporter,data_type))* x[articles] for articles in
possible_articles])


    #Solve and Display
    status=prob.solve()
    #Status, 1:Optimul, 2:not solved, 3:infeasible, 4:unbounded, 5:undef
    print("status:",status)

    #Print the identified lowest article
    for articles in possible_articles:
        if x[articles].value() == 1.0:
            print(articles)

    print("Total Cost of Writing these Articles is: ", value(prob.objective))
```

(.Project_2) PS C:\Users\Aaron\Documents\Project1_5130\Project_1_5130> _5130/tnn_1p.py

Welcome to the CBC MILP Solver

Version: 2.10.3

Build Date: Dec 15 2019

command line - C:\Users\Aaron\Documents\Project1_5130\Project_1_5130\.Pps -timeMode elapsed -branch -printingOptions all -solution C:\Users\Aa

At line 2 NAME MODEL

At line 3 ROWS

At line 17 COLUMNS

At line 3414 RHS

At line 3427 BOUNDS

At line 3683 ENDATA

Problem MODEL has 12 rows, 255 columns and 2631 elements

Coin0008I MODEL read with 0 errors

Option for timeMode changed from cpu to elapsed

Continuous objective value is 551 - 0.00 seconds

Cgl0003I 83 fixed, 0 tightened bounds, 0 strengthened rows, 1 substitut

Cgl0004I processed model has 0 rows, 0 columns (0 integer (0 of which b

Cbc3007W No integer variables - nothing to do

Cuts at root node changed objective from 551 to -1.79769e+308

Probing was tried 0 times and created 0 cuts of which 0 were active aft

Gomory was tried 0 times and created 0 cuts of which 0 were active afte

Knapsack was tried 0 times and created 0 cuts of which 0 were active af

Clique was tried 0 times and created 0 cuts of which 0 were active afte

MixedIntegerRounding2 was tried 0 times and created 0 cuts of which 0 w

FlowCover was tried 0 times and created 0 cuts of which 0 were active a

TwoMirCuts was tried 0 times and created 0 cuts of which 0 were active

ZeroHalf was tried 0 times and created 0 cuts of which 0 were active af

Result - Optimal solution found

Objective value: 551.00000000

Time (Wallclock seconds): 0.00

Option for printingOptions changed from normal to all

Total time (CPU seconds): 0.02 (Wallclock seconds): 0.02

status: 1

('A0', 'A1', 'A2', 'A7')

Total Cost of Writing these Articles is: 551.0